ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 2

Linux и WSL для научных вычислений

Практика NEP-kappa: от установки до результатов

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе

Лекции 1–2
Python и рабочая среда
Linux · WSL · HPC

Лекции 3–4
Численные методы и уравнения
Интегралы · ОДУ/УЧП

Лекции 5–6
Монте-Карло и перенос
Выборки · Диффузия

Лекции 7–8
Машинное обучение в физике
Нейросети · PINN · NEP

Лекция 2 — Linux и вычислительная среда

Подготовим Linux-окружение и рассчитаем теплопроводность 3C-SiC в NEP-kappa.

Практические задачи:

- использовать WSL для работы с Linux в Windows;
- выполнять основные операции в Linux-терминале;
- устанавливать NEP-kappa и запускать расчёт 3C-SiC;
- подключаться к удалённому серверу по SSH;
- отправлять задания в Slurm и получать графики результатов.

План

1. Windows и WSL
2. Основные операции Linux
3. Python в Linux
4. Практика NEP-kappa
5. Сервер и Slurm

1

Windows и WSL

Windows и Linux в научных вычислениях

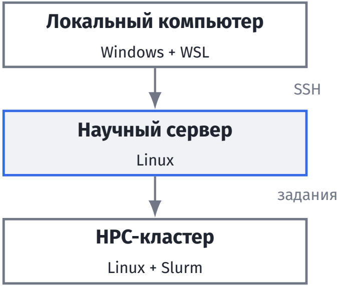
Работа в Windows

Средства разработки и расчётов:

- Python и Jupyter Notebook;
- VS Code и PyCharm;
- MATLAB, C/C++ и другие языки.

Linux на серверах

- научные серверы;
- HPC-кластеры;
- системы пакетных расчётов.



Локальный компьютер

Windows + WSL

SSH

Научный сервер

Linux

задания

HPC-кластер

Linux + Slurm

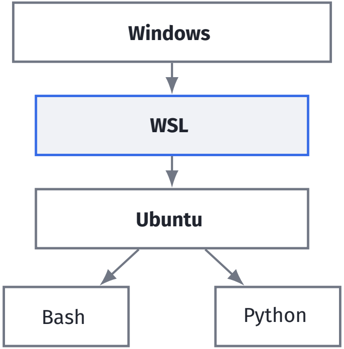
Что такое WSL?

WSL — Windows Subsystem for Linux

WSL позволяет запускать Linux-среду непосредственно внутри Windows.

Внутри WSL доступны:

- дистрибутив Ubuntu;
- терминал Bash;
- Python и компиляторы;
- команды и программы Linux.


Windows
WSL
Ubuntu
Bash
Python

Установка WSL в Windows

Установка

1. Откройте PowerShell или Windows Terminal **от имени администратора**.

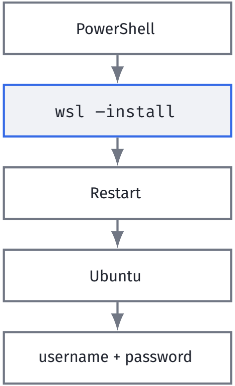
2. Выполните команду:

```
wsl --install
```

3. Перезагрузите Windows, если система попросит.

4. При первом запуске Ubuntu создайте:

- имя Linux-пользователя;
- пароль.



PowerShell

wsl –install

Restart

Ubuntu

username + password

Как открыть WSL?

Через меню «Пуск»

Найдите в меню Windows приложение **Ubuntu** и запустите его.

Через терминал Windows

В PowerShell или Windows Terminal выполните:

```
wsl
```

Проверить установленные дистрибутивы:

```
wsl --list --verbose
```

После входа в Linux

Приглашение командной строки:

```
student@DESKTOP:~$
```

Здесь:

- `student` — имя пользователя;
- `DESKTOP` — имя компьютера;
- `~` — домашний каталог.

Первая команда:

```
pwd
```

Где находятся файлы Windows?

Доступ к файлам Windows

В WSL диски Windows доступны через каталог `/mnt`.

Windows	WSL
`C:\`	`/mnt/c/`
`D:\`	`/mnt/d/`

Например:

```
cd /mnt/c/Users
ls
```

Файлы Linux

Домашний каталог пользователя обозначается символом ~.

```
cd ~
pwd
```

Открыть файлы WSL в Windows

Текущий каталог WSL можно открыть в Проводнике Windows:

```
explorer.exe .
```

2

Основные операции Linux

Терминал: управление через команды

Терминал принимает команды и выводит результат их выполнения.

Вопрос	Команда
Где я?	`pwd`
Что здесь?	`ls`
Куда перейти?	`cd`

Общий вид команды

```
команда [опции] [аргументы]
```

- `команда` — что выполнить;
- `опции` — как выполнить;
- `аргументы` — с чем выполнить действие.

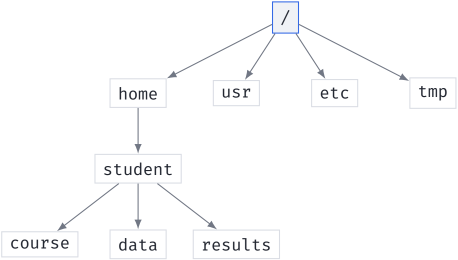
Например:

```
ls -l /home/student
```

Файловая система Linux

Иерархическая структура

В Linux все файлы и каталоги образуют одно дерево, которое начинается с корневого каталога /.


/
home
usr
etc
tmp
student
course
data
results

Полезные обозначения

/	корневой каталог
~	домашний каталог
.	текущий каталог
..	родительский каталог

Пути

Абсолютный:

`/home/student/course`

Относительный:

`course/data`

Создание каталогов и файлов

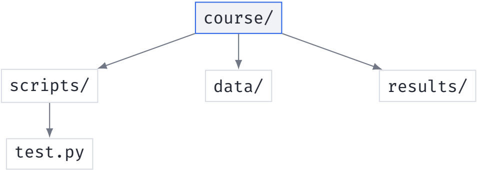
```
mkdir course
cd course

mkdir scripts
mkdir data
mkdir results

touch scripts/test.py
```

Проверим структуру:

```
ls
ls scripts
```


course/
scripts/
data/
results/
test.py

Две команды

`mkdir` — создаёт каталог.

`touch` — создаёт файл, если его ещё нет.

Навигация: `pwd`, `ls`, `cd`

Посмотреть, где мы находимся

```
pwd
```

Посмотреть содержимое каталога

```
ls
ls -l
ls -a
```

- `-l` — подробная информация;
- `-a` — включая скрытые файлы.

Перейти в другой каталог

```
cd course
cd ..
cd ~
cd /home/student/course
```

Пример работы

```
pwd
ls
cd course
```

Копирование, перемещение и удаление

Копирование: `cp`

```
cp test.py test_backup.py
```

Перемещение и переименование: `mv`

```
mv test.py scripts/
```

Переименование:

```
mv old.py new.py
```

Удаление файла: `rm`

```
rm file.txt
```

Удаление каталога

```
rm -r folder
```

Опция `-r` удаляет каталог вместе с его содержимым.

Осторожно

`rm` удаляет файлы без перемещения в корзину.

Просмотр и поиск в файлах

Просмотр содержимого

```
cat result.txt
less result.txt
```

`cat` выводит файл целиком, а `less` удобен для просмотра больших файлов.

Начало и конец файла

```
head result.txt
tail result.txt
```

Поиск текста: `grep`

Найти строки, содержащие слово:

```
grep "error" simulation.log
```

Без учёта регистра:

```
grep -i "error" simulation.log
```

Мини-шпаргалка Linux

Действие	Команда	Пример
Текущий каталог	`pwd`	`pwd`
Список файлов	`ls`	`ls -l`
Перейти в каталог	`cd`	`cd results`
Создать каталог	`mkdir`	`mkdir data`
Создать файл	`touch`	`touch test.py`
Копировать	`cp`	`cp a.txt b.txt`
Переместить / переименовать	`mv`	`mv a.txt data/`
Удалить	`rm`	`rm a.txt`
Просмотреть файл	`cat, less`	`less output.log`
Найти текст	`grep`	`grep ”error”log.txt`

3

Python в Linux

Установка Miniconda в WSL

Скачивание

```
wget https://repo.anaconda.com/
    miniconda/Miniconda3-latest-Linux-
    x86_64.sh
```

Установка

```
bash Miniconda3-latest-Linux-x86_64.sh
```

Следуйте инструкциям установщика.

После установки

Перезапустите терминал или выполните:

```
source ~/.bashrc
```

Проверим conda:

```
conda --version
```

Создание Python-окружения с conda

Создание окружения

Создадим отдельное окружение для курса:

```
conda create -n numcalc python=3.12
```

Активируем его:

```
conda activate numcalc
```

Выход из окружения

```
conda deactivate
```

Проверка

После активации:

```
which python
python --version
```

Путь будет связан с Miniconda, например:

```
~/miniconda3/envs/numcalc/bin/
    python
```

В окружении numcalc установлены свой Python и свои библиотеки.

Установка научных библиотек

Установка

Сначала активируем окружение:

```
conda activate numcalc
```

Затем установим библиотеки:

```
conda install numpy scipy matplotlib
```

Проверка

```
python
```

В Python:

```
import numpy as np
print(np.__version__)
```

Посмотреть окружения

```
conda env list
```

Запускаем Python-расчёт в WSL

Файл `scripts/calculation.py`

```python
import numpy as np
x = np.linspace(0, 1, 5)
y = x ** 2
print(y)
```

Структура проекта

```
course/
|-- scripts/
|   '-- calculation.py
|-- data/
`-- results/
```

Запуск

Перейдём в каталог проекта:

```
cd ~/course
```

Активируем окружение:

```
conda activate numcalc
```

Запустим программу:

```
python scripts/calculation.py
```

Сохраняем результаты расчёта

Перенаправление вывода

Перезаписать файл вывода:

```
python scripts/calculation.py \
  > results/output.txt
```

Добавить вывод в конец файла:

```
python scripts/calculation.py \
  >> results/output.txt
```

Просмотр результата

Показать содержимое файла:

```
cat results/output.txt
```

Для длинного файла:

```
less results/output.txt
```

Последние строки:

```
tail results/output.txt
```

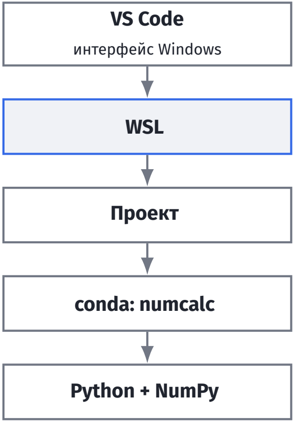
VS Code + WSL + conda

Рабочий процесс

1. VS Code установлен в Windows;
2. установлено расширение **WSL**;
3. проект хранится внутри WSL;
4. активируется conda-окружение;
5. Python запускается внутри Linux.

Откроем проект из WSL:

```
cd ~/course
conda activate numcalc
code .
```



VS Code
интерфейс Windows

WSL

Проект

conda: numcalc

Python + NumPy

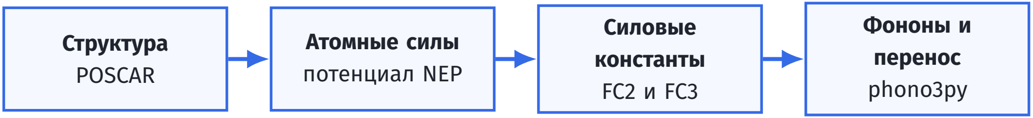
4

Практика NEP-kappa

Что будем рассчитывать в NEP-kappa

3C-SiC — кубический карбид кремния.

Получим фононные свойства и зависимость решёточной теплопроводности от температуры.



Структура
POSCAR

Атомные силы
потенциал NEP

Силовые константы
FC2 и FC3

Фононы и перенос
phono3py

Входные данные

Структура, готовая модель NEP и параметры расчёта в YAML.

Результаты

Файлы расчёта, графики и краткий отчёт.

1. Создаём окружение NEP-kappa

В терминале Linux / WSL

```
conda create -n nepkappa python=3.11
conda activate nepkappa

python --version
which python
```

Что проверить

В приглашении появилось `(nepkappa)`.

Версия Python — 3.11; путь указывает на окружение `nepkappa`.

При следующем входе в терминал

Окружение создают один раз. Перед работой выполняют `conda activate nepkappa`.

2. Получаем код и устанавливаем программу

```
cd ~
git clone https://github.com/lyushisyan/NEP-kappa.git
cd NEP-kappa
python -m pip install -e .

nepkappa --version
nepkappa --help
```

Где выполнять установку

В каталоге с файлом `pyproject.toml`, после активации окружения.

Если скачали ZIP

Распакуйте архив и перейдите в `NEP-kappa-main`. Команда установки та же.

3. Готовим три входных файла

```
mkdir -p ~/work/3C-SiC
cd ~/work/3C-SiC
src=~/NEP-kappa
cp "$src/examples/nep-rta-wigner-3ph.yaml" input.yaml
cp "$src/examples/structures/3C-SiC/POSCAR_primitive" POSCAR
cp "$src/potentials/3C-SiC/nep_3C-SiC.txt" .
ls
```

`POSCAR`
Кристаллическая структура

`nep_3C-SiC.txt`
Модель потенциала

`input.yaml`
Параметры расчёта

После загрузки ZIP: `src=~/NEP-kappa-main`.

4. Настраиваем пути в input.yaml

```yaml
structure:
  poscar: POSCAR
  relaxation: true

calculator:
  name: nep
  model: nep_3C-SiC.txt

output:
  result-dir: results
```

Открыть файл

```
nano input.yaml
```

`relaxation: true` — сначала оптимизировать структуру.

`model` — путь к модели NEP.

Результаты будут в `results/`.

Сохранить: **Ctrl+O**, затем Enter.
Выйти: **Ctrl+X**.

Остальные разделы шаблона сохраняем. Все команды расчёта выполняем из `~/work/3C-SiC`.

5. Параметры расчёта 3C-SiC

```
force-constant:
  dim-fc2: [3, 3, 3]
  dim-fc3: [3, 3, 3]

kappa:
  engine: phono3py
  mesh: [31, 31, 31]
  temps: [100, 1000, 50]
  method: rta
  isotope: true
  wigner: true
```

Размеры и температуры

- Суперячейки FC2 и FC3: $3 \times 3 \times 3$.
- Фононная сетка: $31 \times 31 \times 31$.
- Температуры: от 100 до 1000 K, шаг 50 K.

Модель переноса

RTA — приближение времени релаксации. Включены изотопное рассеяние и вклад когерентности в формализме Вигнера.

Для исследования материала отдельно проверяют сходимость по суперячейке и сетке.

6. Проверяем параметры и запускаем расчёт

Сначала проверка

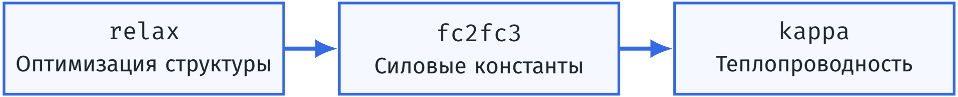
```
nepkappa info input.yaml
```

Показывает прочитанные параметры; расчёт не запускается.

Затем запуск в Linux / WSL

```
nepkappa run input.yaml
```

На кластере эту команду выполняют через Slurm.



`relax`
Оптимизация структуры

`fc2fc3`
Силовые константы

`kappa`
Теплопроводность

`run` выполняет эти три этапа. Графики строятся отдельной командой.

Как следить за расчётом

Фрагмент вывода из руководства

```
[Plan] relax -> fc2fc3 -> kappa
[Step 1] Relax Structure
[Step 2] Generate Force Constants
[Step 3] Compute Kappa with Phono3py CLI
[Main] Completed: run
Run log saved to: results/run.log
```

Журнал в другом терминале

```
cd ~/work/3C-SiC
tail -f results/run.log
```

Ctrl+C завершает просмотр журнала.

Подробный вывод при запуске

```
nepkappa run input.yaml -v
```

`Completed: run` означает успешное завершение расчёта.

7. Проверяем файлы результатов

```
ls -lh results
```

Файл или каталог	Содержимое
`POSCAR_relaxed`	Оптимизированная структура
`fc2.hdf5`, `fc3.hdf5`	Силовые константы
`kappa-m313131.hdf5`	Данные теплопереноса
`force-jobs/`	Смещения и рассчитанные силы
`run.log`	Полный журнал запуска
`provenance.yaml`	Параметры, версии и статус расчёта

8. Строим графики и создаём отчёт

Раздел plot в input.yaml

```
plot:
  layout: both
  path: seekpath
  tau: nu
  temperature: 300
  kappa: all
```

`both` — отдельные рисунки и общий рисунок из шести панелей.

После завершения run

```
nepkappa plot input.yaml
nepkappa report input.yaml
```

Где искать

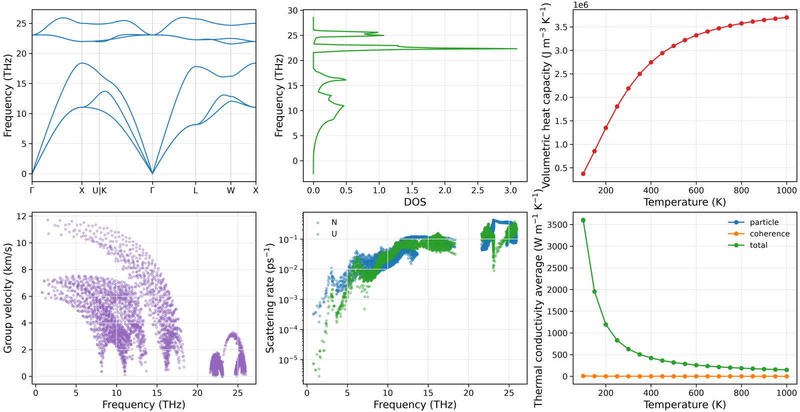
`results/plots/combined.png`

`results/report.md`

`results/report.yaml`

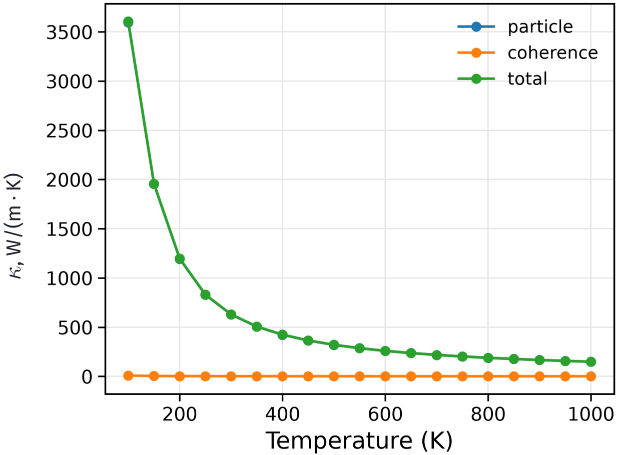
Используются уже рассчитанные данные.

Результат: фононные свойства и теплоперенос



3C-SiC: составной рисунок из главы 3 руководства NEP-kappa.

Теплопроводность 3C-SiC: читаем график



Оси

Температура — K.
Теплопроводность — W/(m · K).

Кривые

`particle` — фононный вклад;

`coherence` — вклад когерентности;

`total` — их сумма.

В этом примере теплопроводность уменьшается с ростом температуры.

Повторный расчёт: меняем температуры

В разделе kappa

```
  temps: [300, 600, 100]
```

Новый диапазон: 300, 400, 500 и 600 K.

Сохраняем структуру, потенциал и параметры силовых констант.

Повторяем этап переноса

```
nepkappa info input.yaml
nepkappa kappa input.yaml
nepkappa plot input.yaml
nepkappa report input.yaml
```

Используем готовые `fc2.hdf5`, `fc3.hdf5` и `phono3py_disp.yaml`.

Если нужны оба набора результатов, сначала скопируйте каталог расчёта.

5

Сервер и Slurm

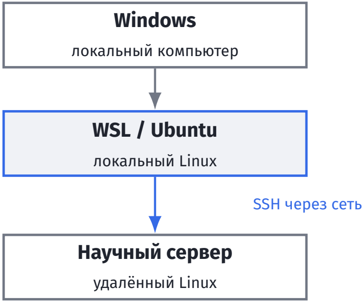
Локальная и удалённая работа

Работа в WSL

- работать с Linux-командами;
- использовать файловую систему;
- создавать conda-окружения;
- запускать Python-программы;

Среда на сервере

- Linux;
- терминал Bash;
- Python / conda;
- те же команды и структура файлов.



Windows

локальный компьютер

WSL / Ubuntu

локальный Linux

SSH через сеть

Научный сервер

удалённый Linux

Подключение к серверу через SSH

SSH — Secure Shell

SSH позволяет открыть терминал удалённого Linux-компьютера через сеть.

Общий формат:

```
ssh username@server
```

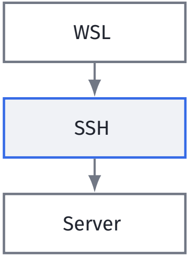
После подключения проверим:

```
whoami
hostname
pwd
ls
```

Где выполняются команды

До подключения: локально в WSL.

После подключения: на сервере.



WSL

SSH

Server

Передаём исходный код и скачиваем рисунок

Команды выполняются в локальном терминале.

Если сервер не имеет доступа к GitHub

```
scp ~/Downloads/NEP-kappa-main.zip user@server:~/
```

На сервере: `unzip NEP-kappa-main.zip`, затем установка из корня проекта.

После расчёта скачиваем составной рисунок

```
scp user@server:~/work/3C-SiC/results/plots/combined.png \
    ~/Downloads/
```

`user@server` замените своим именем пользователя и адресом сервера.

Планировщик задач Slurm

Один сервер

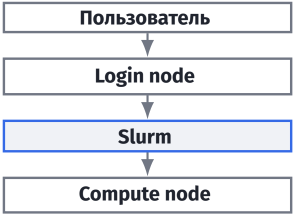
Пользователь может самостоятельно запустить программу:

User → Program → CPU

HPC-кластер

На кластере одновременно работают:

- много пользователей;
- много задач;
- сотни или тысячи CPU;
- GPU и другие ресурсы.



Пользователь

Login node

Slurm

Compute node

Slurm определяет:

- когда запустить задачу;
- на каком вычислительном узле;
- сколько CPU, памяти или GPU выделить.

Расчёты запускают на вычислительных узлах через очередь Slurm.

Slurm: запрашиваем ресурсы

Начало файла submit.slurm

```
#!/bin/bash
#SBATCH --job-name=nepkappa
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --time=02:00:00
#SBATCH --output=nepkappa-%j.out
```

Для этого примера

Один узел, одна задача, 8 CPU.

Память — 16 GB.

Лимит времени — 2 часа.

`%j` заменяется номером задания в имени файла вывода.

Сохраните файл в `~/work/3C-SiC`. Параметры ресурсов, `–partition` и `–account` зависят от кластера.

Slurm: окружение и запуск NEP-kappa

Продолжение того же файла submit.slurm

```
set -e
cd "$SLURM_SUBMIT_DIR"
# Adjust this path for your installation of Conda.
source ~/miniconda3/etc/profile.d/conda.sh
conda activate nepkappa

export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export RAYON_NUM_THREADS="$SLURM_CPUS_PER_TASK"
export OPENBLAS_NUM_THREADS=1
export MKL_NUM_THREADS=1

srun --cpus-per-task="$SLURM_CPUS_PER_TASK" nepkappa run input.yaml
```

Путь `~/miniconda3` замените своим, например `~/miniforge3`. Число потоков согласовано с выделенными CPU.

Slurm: отправляем и проверяем задание

В каталоге расчёта

```
cd ~/work/3C-SiC
sbatch submit.slurm
squeue -u "$USER"
```

Отмена по номеру задания

```
scancel 123456
```

Замените `123456` своим номером.

Состояния в очереди

PD — ожидает ресурсов.

R — выполняется.

После завершения

Проверяем `nepkappa-ID.out`, `results/run.log` и файлы результатов.

Затем выполняем `plot` и `report`.

После успешной отправки пакетного задания можно закрыть SSH-соединение.

Итоги лекции

Linux и окружение

Файлы и каталоги, команды терминала, отдельное окружение Conda.

NEP-kappa

Подготовка 3C-SiC: структура, модель NEP и YAML.
Проверка, расчёт, графики и отчёт.

Сервер и Slurm

SSH, передача файлов, отправка задания и просмотр результатов.

GitHub `github.com/lyushisyan/NEP-kappa`